CodeQuest

Operations on Data Structures

Python Programming

CodeQuest

Operations on - Set

Python Programming

CodeQuest

Set - Operations

A set in Python is an unordered collection of unique elements, which are mutable.

Creating a Set:

Example 01: **#By using curly braces { }**

```
s ={10,20,30,40}
print(s) # output: {10,40,30,20}
```

Example 02: **#By using set() function along-with list**

```
l = [10,20,30,40,10,20,10]
s = set(l)
print(s) # It will create set with elements {10,20,30,40} after eliminating duplicate values.
```

Example 03: **# By using set() function along-with range() function**

```
s =set(range(5))
print(s) # {0,1,2,3,4}
```

CodeQuest

Set - Operations

While creating an empty set, we have to use the set() function.

Example 04: **#By using curly braces { } for an empty set (Not recommended)**

```
s={}  # It is treated as a dictionary but not an empty set.
print(s) # { }
type(s) # <class 'dict'>
```

Example 05: **#By using set() function for an empty set**

```
s = set() # It will create an empty set
print(s) # { }
type(s) # <class 'set'>
```

CodeQuest

Set - Operations

Operations on Set:

The following functions can be applied to a set.

1. add()
2. update()
3. copy()
4. pop()
5. remove()
6. discard()
7. clear()
8. union()
9. intersection()
10. difference()
11. symmetric_difference

CodeQuest

Set - Operations

1) add() function :

The add() function is used to add an element to a set.

Syntax:

```
set.add(element)
```

Example:

```
my_set = {1, 2, 3}        # Create a set
my_set.add(4)             # Add an element to the set
print(my_set)             # Output: {1, 2, 3, 4}
```

Key Features:

1. Adds a Single Element: The add() method adds a single element to the set.
2. No Duplicates: If the element already exists in the set, it will not be added again.
3. Returns None: The add() method returns **None.**

Set - Operations

2) update() function :

The update() function in Python is used to add **multiple** elements to a set. Arguments are not individual elements, and these are Iterable objects like List, range, etc.

Syntax:

```
set.update(iterable)
```

Example:

```
s={10,20,30}
l=[40,50,60,10]
s.update(l,range(5))
print(s)      # Output:  {0, 1, 2, 3, 4, 40, 10, 50, 20, 60, 30}
```

Set - Operations

3) copy() function :

The copy() function in Python is used to create a shallow copy (only copies the references to the elements) of a set.

Syntax:

```
s=set.copy()
```

Example:

```
s={1,2,3}
my_set_copy = s.copy()      # Create a copy of the set
print(my_set_copy)          # {1, 2, 3}
my_set_copy.add(5)
print(my_set_copy)          # {1,2,3,5}
print(s)                    # {1,2,3}
```

Note: Any changes made to the copied set will have no impact on the original set.

CodeQuest

Set - Operations

4) pop() function :

The pop() function in Python is used to remove and return an arbitrary element from a set.

Syntax:

```
set.pop()
```

Example:

```
s={1,2,3}
popped_element = s.pop()   # Remove and return an arbitrary element from the set
print(popped_element)              # Output: 1
print(s)  # Output: {2, 3}
```

Note: If the set is empty, the pop() function raises a **KeyError**.

Set - Operations

5) remove() function :

The remove() function in Python is used to remove a specified element from a set.

Syntax:

```
set.remove(element)
```

Example:

```
my_set={1,2,3}
my_set.remove(2)        # Remove specified element ( i.e. 2 ) from the set
print(my_set)           # Output: {1, 3}
my_set.remove(9)        # Raises KeyError
```

Note: It raises a **KeyError** if the specified element is not found.

Set - Operations

6) discard() function :

The discard() function in Python is used to remove a specified element from a set.

Syntax:

```
set.discard(element)
```

Example:

```
my_set={1,2,3}
my_set.discard(2)          # Remove specified element from the set
print(my_set)              # Output: {1, 3}
```

Note: It will **not** raise **KeyError** if the specified element is not found.

Set - Operations

7) clear() function :

The clear() function in Python is used to remove all elements from a set.

Syntax:

```
set.clear()
```

Example:

```
my_set={1,2,3}
my_set.clear()          # Remove all elements from the set
print(my_set)           # Output: {}
```

Note: The clear() function does not raise an error, even if the set is already empty.

Set - Operations

8) union() function :

The union() function in Python is used to return all elements present in both sets.

Syntax:

```
x.union(y)
```

Example:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.union(y))        #{10, 20, 30, 40, 50, 60}
print(x|y)               #{10, 20, 30, 40, 50, 60}
```

Set - Operations

9) intersection() function :

The intersection() function in Python is used to return common elements present in both sets.

Syntax:

```
x.intersection(y)
```

Example:

```
x={10,20,30,40}
y={30,40,50,60}
print(x.intersection(y))        #{30, 40}
print(x&y)                      #{30, 40}
```

Set - Operations

10) difference() function :

The difference() function in Python is used to find the elements that are in one set but not in another.

Syntax:

```
set1.difference(set2)
```

Example:

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
diff = set1.difference(set2)  # Finds the elements in set1 but not in set2
print(diff)                   # Output: {1, 2}
```

Set - Operations

11) symmetric_difference() function :

The symmetric_difference() function in Python is used to find the elements that are in **either** of two sets, but **not in** their **intersection**.

Syntax:

```
set1. symmetric_difference (set2)
```

Example:

```
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}
sym_diff = set1.symmetric_difference(set2)
print(sym_diff)                    # Output: {1, 2, 5, 6}
```

CodeQuest

Set Comprehension

Set Comprehension :

Set comprehension in Python is a concise way to create sets from existing iterables or other sets.

Syntax:

```
{expression for variable in iterable}
```

Example:

```
numbers = {x for x in range(1, 6)}
print(numbers)  # Output: {1, 2, 3, 4, 5}
```

Note: Set comprehension can be used with various iterables, such as lists, tuples, and other sets.

CodeQuest

Thank You

CodeQuest

Operations on Data Structures

Python Programming

CodeQuest

Operations on - Dictionary

Python Programming

CodeQuest

Dictionary

We can use List, Tuple, and Set to represent a group of individual objects as a **single entity**.

If we want to represent a **group of objects** as **key-value pairs**, then we should go for a Dictionary.

Characteristics :

- **Duplicate keys** are **not allowed**
- Duplicate **values** are **allowed**.
- **Heterogeneous** objects are allowed for both keys and values.
- The insertion order is **not** preserved.
- Dictionaries are mutable, dynamic.
- indexing and slicing concepts are **not applicable.**

CodeQuest

Dictionary

How to create a Dictionary?

Example 01: #Creating an empty dictionary

```
d={}  or d=dict()
print(d) # { }
```

Example 02: #We can add entries into the dictionary as follows:

```
d[100]=“Kiran"
d[200]=“Prateek"
d[300]=“Sunil"
print(d)   #{100: ‘Kiran', 200: ‘Prateek', 300: ‘Sunil’}
```

Example 03: #We can also create a dictionary as follows:

```
d={100:’Kiran' ,200:’Prateek', 300:’Sunil’}
print(d)   #{100: ‘Kiran', 200: ‘Prateek', 300: ‘Sunil’}
```

Dictionary

How to access data from the dictionary?

We can access data by using keys.

```
d={10: 'ram' ,20:'ravi', 30:’shyam'}
print(d[10])     #ram
print(d[30])     #shyam
```

If the specified key is not available, then we will get KeyError

```
print(d[40])  # KeyError: 40
```

Dictionary

How to update the dictionary?

We can update the dictionary by adding one or more elements using the following syntax.

Syntax: d[key] = value

If the key is not available, then a new entry will be added to the dictionary with the specified key-value pair.

If the key is already available, then the old value will be replaced with the new value.

```
d={10: 'ram' ,20:'ravi', 30:'shyam'}
d[40]='suraj'      #Adds a new key-value pair to the dictionary
print(d)           # {10: 'ram' ,20:'ravi', 30:'shyam', 40: 'suraj'}

d[20]='rohit'      # As the key (20) already exist, it updates the value associated with key 20
print(d)           # {10: 'ram' ,20:'rohit', 30:'shyam', 40: 'Suraj'}
```

Dictionary

How to delete elements from the dictionary?

We can delete an element from the dictionary by using the **del** keyword.

Syntax: del d[key]

It deletes the entry associated with the specified key. If the key is not available, then we will get **KeyError**.

```
d={10: 'ram' ,20:'ravi', 30:'shyam’}
del d[20]
print(d)            # {10: 'ram' , 30:'shyam’}

del d[70]           # KeyError: 70
```

CodeQuest

Dictionary - Operations

Operations on Dictionary:

The following functions can be applied to a dictionary.

1. **len()**
2. **get()**
3. **pop()**
4. **popitem()**
5. **keys()**
6. **values()**
7. **items()**
8. **copy()**
9. **setdefault()**
10. **update()**

CodeQuest

Dictionary - Operations

1) len() function :

The len() function is used to get the number of items (key-value pairs) in a dictionary.

Syntax:

```
len(dictionary)
```

Example:

```
person = {"name": "John", "age": 30, "city": "New York"}
num_items = len(person)
print(num_items)       # Output: 3
```

CodeQuest

Dictionary - Operations

2) get() function :

The get() function is used to retrieve the value associated with a specified key.

Syntax:

dictionary. get(key, default=None)

Example 01:

```
person = {"name": "John", "age": 30}
print(person.get("name"))          # Output: John
print(person.get("country"))       # Output: None
```

Example 02:

```
d={100:"disha",200:"raj",300:"arya"}
print(d.get(100,"Guest")) # Output: disha
print(d.get(500,"Guest"))  # Output: Guest
```

Dictionary - Operations

3) pop() function :

The pop() function removes the entry associated with the specified key and returns the corresponding value.

If the specified key is not available, then it will raise **KeyError**.

Syntax:

```
dictionary.pop(key)
```

Example:

```
d={100:"disha",200:"raj",300:"arya"}
print(d.pop(100))      #Output: disha
print(d.pop(500))      #Output: KeyError:500
```

Dictionary - Operations

4) popitem() function :

The popitem() function removes an **arbitrary** item (key-value) from the dictionary and returns it.

If the dictionary is **empty**, then it will raise a **KeyError** exception.

Syntax:

```
dictionary.popitem()
```

Example 01:

```
d={100:"disha",200:"raj",300:"arya"}
print(d.popitem())      #Output: (300:arya)
```

Example 02:

```
d={}
print(d.popitem())      #Output: KeyError: dictionary is empty
```

CodeQuest

Dictionary - Operations

5) keys() function :

The keys() function returns all keys associated with a dictionary.

Syntax:

```
dictionary.keys()
```

Example:

```
d={100:"disha",200:"raj",300:"arya"}
print(d.keys())     #Output: dict_keys([100,200,300])
for k in d.keys():  #Displays all the keys present in the dictionary
    print(k)

#Output:   100
           200
           300
```

CodeQuest

Dictionary - Operations

6) values() function :

The values() function returns all values associated with a dictionary.

Syntax:

```
dictionary.values()
```

Example:

```
d={100:"disha",200:"raj",300:"arya"}
print(d.values()) #Output: dict_values(['disha','raj','arya'])
for v in d.values():  #Displays all the keys present in the dictionary
    print(v)
```

```
#Output:  disha
          raj
          arya
```

CodeQuest

Dictionary - Operations

7) items() function :

The items() function returns a list of tuples representing key-value pairs.

[(k,v),(k,v),(k,v)]

Syntax:

dictionary.**items**()

Example :

```
d={100:"disha",200:"raj",300:"arya"}
for k,v in d.items():
    print(k, " - ", v)
```

#Output:

```
100 - disha
200 - raj
300 - arya
```

CodeQuest

Dictionary - Operations

8) copy() function :

The copy() function is used to create a shallow copy of a dictionary.

Syntax:

```
dictionary.copy()
```

Example:

```
person = {"name": "John", "age": 30}
p_copy = person.copy()
print(p_copy)  # Output: {'name': 'John', 'age': 30}
```

Dictionary - Operations

9) setdefault() function :

The setdefault() function is used to **set a default value** for a key if **it does not exist** in the dictionary.

Syntax:

```
dictionary.setdefault(key, default=None)
```

Example:

```
person = {"name": "John", "age": 30}
country = person.setdefault("country", "USA")
print(country)          # Output: USA
print(person)           # Output: {'name': 'John', 'age': 30, 'country': 'USA'}
```

Dictionary - Operations

10) update() function :

The update() function is used to **update** a dictionary **with items** from another dictionary or an **iterable** of key-value pairs.

Syntax:

```
dictionary.update(other)
```

Example:

```
person = {"name": "John", "age": 30}
info = {"country": "USA", "city": "New York"}
person.update(info)
print(person)
```

 # Output: {'name': 'John', 'age': 30, 'country': 'USA', 'city': 'New York}

CodeQuest

Dictionary Comprehension

Dictionary comprehension in Python is a concise way to create dictionaries from existing iterables or other dictionaries.

Syntax:

```
{key: value for variable in iterable}
```

Example:

```
fruits = ["apple", "banana", "cherry"]
fruit_dict = {fruit: len(fruit) for fruit in fruits}
print(fruit_dict)          # Output: {'apple': 5, 'banana': 6, 'cherry': 6}
```

CodeQuest

Thank You